

Buổi 9: Storage, xử lý sự kiện, thao tác...
1. DOM là gì?
2. Truy cập phần tử: getElementB...
3. Thay đổi nội dung: innerText, in...
4. Gắn sự kiện: onclick, addEvent...
5. Storage (localStorage, Session...

Expand all
Back to top
Go to bottom

Buổi 9: Storage, xử lý sự kiện, thao tác DOM

DOM là gì?

Truy cập phần tử: `getElementById`,
`getElementsByClassName`,`getElementsByName`,`querySelector`,`querySelectorAll`
Thay đổi nội dung: `innerText`, `innerHTML`, `value`
Gắn sự kiện: `onclick`,`addEventListener`
Storage (`localStorage`, `sessionStorage`, `cookies`)

1. DOM là gì?

1. Khái niệm DOM

DOM (Document Object Model) là mô hình dữ liệu dạng cây biểu diễn cấu trúc của một tài liệu HTML hoặc XML.

Trình duyệt chuyển nội dung HTML thành DOM để JavaScript có thể đọc, chỉnh sửa hoặc thao tác với các phần tử HTML.

Ví dụ:

```
<html>  
<body>  
  <h1>Hello</h1>  
</body>  
</html>
```

→ DOM sẽ là một cây đối tượng mà trong đó:

- `document` là gốc
- `html`, `body`, `h1` là các node con

2. Cấu trúc DOM

- `document`: đại diện toàn bộ trang HTML.
- Các phần tử HTML: đại diện bằng các `Element Nodes`
- Nội dung văn bản: đại diện bằng `Text Nodes`
- Mỗi node có thể có: cha (`parentNode`), con (`childNodes`), anh em (`nextSibling`, `previousSibling`).

3. Các thao tác DOM cơ bản

a. Truy cập phần tử

☛ Các phương thức phổ biến:

Phương thức	Mô tả
<code>getElementById(id)</code>	Trả về phần tử có id cụ thể
<code>getElementsByClassName(class)</code>	Trả về danh sách các phần tử có class
<code>getElementsByTagName(tag)</code>	Trả về danh sách phần tử theo tag
<code>querySelector(selector)</code>	Trả về phần tử đầu tiên khớp với selector
<code>querySelectorAll(selector)</code>	Trả về danh sách tất cả phần tử khớp

Ví dụ:

```
<p id="demo">Xin chào</p>  
<script>  
  const el = document.getElementById("demo");  
  console.log(el.textContent); // "Xin chào"  
</script>
```

b. Thay đổi nội dung & thuộc tính DOM

Các thao tác:

Thuộc tính/phương thức	Mô tả
<code>element.textContent</code>	Thay đổi nội dung văn bản
<code>element.innerHTML</code>	Thay đổi HTML bên trong
<code>element.style.color = 'red'</code>	Thay đổi CSS
<code>element.setAttribute(name, value)</code>	Thay đổi thuộc tính
<code>element.classList.add/remove()</code>	Thêm/xóa class

Ví dụ:

```
const el = document.getElementById("demo");  
el.textContent = "Chào các bạn!";  
el.style.color = "blue";
```

c. Thêm/Xóa phần tử DOM

☛ Tạo phần tử mới

```
const p = document.createElement("p");  
p.textContent = "Đây là đoạn mới";  
document.body.appendChild(p);
```

☛ Xóa phần tử

```
const el = document.getElementById("demo");  
el.remove(); // hoặc el.parentNode.removeChild(el)
```

4. Gắn sự kiện với DOM

Cách thêm sự kiện

```
const btn = document.getElementById("myBtn");  
btn.addEventListener("click", () => {  
  alert("Bấm vào click!");  
});
```

Bài tập:

Tạo một trang HTML hiển thị lời chào, và một nút.

Khi nhấn nút, nội dung lời chào sẽ thay đổi.

2. Truy cập phần tử: `getElementById`, `querySelector`, `querySelectorAll`

1. `getElementById(id)`

- Trả về phần tử đầu tiên có id được chỉ định
- Kết quả: 1 phần tử
- Phổ biến và nhanh nhất nếu chỉ cần 1 phần tử

☛ Ví dụ:

```
<p id="demo">Xin chào</p>  
<script>  
  const el = document.getElementById("demo");  
  el.style.color = "red";  
</script>
```

2.2. `getElementsByClassName(class)`

Trả về một `HTMLCollection` (gần giống mảng) chứa tất cả phần tử có class

Cần dùng vòng lặp để xử lý từng phần tử

☛ Ví dụ:

```
<p class="item">Item 1</p>  
<p class="item">Item 2</p>  
<script>  
  const items = document.getElementsByClassName("item");  
  for (let i = 0; i < items.length; i++) {  
    items[i].style.color = "blue";  
  }  
</script>
```

2.3. `getElementsByName(tag)`

- Trả về một `HTMLCollection` chứa tất cả phần tử theo tag (`div`, `p`, `h1`...)
- Dùng khi muốn truy cập theo thẻ

☛ Ví dụ:

```
<h2>Tiêu đề 1</h2>  
<h2>Tiêu đề 2</h2>  
<script>  
  const titles = document.getElementsByTagName("h2");  
  for (let title of titles) {  
    title.style.fontSize = "24px";  
  }  
</script>
```

2.4. `querySelector(selector)`

- Trả về phần tử đầu tiên khớp với selector (CSS style)
- Dùng được với `#id`, `class`, `tag`, `div > p`, v.v.

☛ Ví dụ:

```
<p class="msg">Tin nhắn 1</p>  
<p class="msg">Tin nhắn 2</p>  
<script>  
  const firstMsg = document.querySelector(".msg");  
  firstMsg.style.fontWeight = "bold";  
</script>
```

2.5. `querySelectorAll(selector)`

- Trả về `NodeList` (gần giống mảng) chứa tất cả phần tử khớp selector
- Dùng được với `forEach`

☛ Ví dụ:

```
<ul>  
  <li>Apple</li>  
  <li>Orange</li>  
</ul>  
<script>  
  const listItems = document.querySelectorAll("ul li");  
  listItems.forEach(item => {  
    item.style.color = "green";  
  });  
</script>
```

3. So sánh các phương thức

Phương thức	Trả về	Selector kiểu gì	Ghi chú
<code>getElementById</code>	1 phần tử	Id	Nhanh nhất, chỉ dùng được với id
<code>getElementsByClassName</code>	<code>HTMLCollection</code>	<code>..class</code>	Dùng vòng lặp xử lý
<code>getElementsByName</code>	<code>HTMLCollection</code>	<code>div</code> , <code>p</code> , <code>li</code> , ...	Dùng vòng lặp xử lý
<code>querySelector</code>	1 phần tử đầu tiên	CSS selector	Linh hoạt, cú pháp CSS
<code>querySelectorAll</code>	<code>NodeList</code> (nhiều)	CSS selector	Dùng <code>forEach</code> được

4. Bài tập thực hành đơn giản

Đề bài: Đổi màu chữ của từng nhóm phần tử

```
<h1 id="main-title">Chào mừng</h1>  
<p class="note">Lưu ý 1</p>  
<p class="note">Lưu ý 2</p>  
<ul>  
  <li>Item A</li>  
  <li>Item B</li>  
</ul>
```

3. Thay đổi nội dung: `innerText`, `innerHTML`, `value`

1. Giới thiệu chung

- Khi truy cập một phần tử HTML qua JavaScript, ta có thể thay đổi:
- Nội dung hiển thị (text)
- Nội dung HTML bên trong
- Giá trị nhập liệu

2. `innerText` – Lấy hoặc thay đổi văn bản hiển thị

Chỉ lấy/chỉnh nội dung dạng text

Bỏ qua các thẻ HTML bên trong

Không xử lý nội dung ẩn bằng CSS (`display: none`)

☛ Ví dụ:

```
<p id="para">Hello <b>world</b></p>  
<script>  
  const el = document.getElementById("para");  
  console.log(el.innerText); // "Hello world"  
  el.innerText = "Chào bạn!";  
</script>
```

3. `innerHTML` – Lấy hoặc thay đổi HTML bên trong

- Giữ nguyên các thẻ HTML con
- Dùng để thêm thẻ mới, định dạng HTML, tạo bảng,...

☛ Ví dụ:

```
<div id="box">Hello</div>  
<script>  
  const el = document.getElementById("box");  
  el.innerHTML = "<p>Xin chào</p> học viên</p>";  
</script>
```

▲ Cảnh báo:

Không dùng `innerHTML` để chèn dữ liệu người dùng nhập → dễ bị XSS (cross-site scripting)

4. `value` – Lấy hoặc thay đổi giá trị trong input / form

- Áp dụng với: `<input>`, `<textarea>`, `<select>`, ...
- Dùng để đọc/ghi giá trị nhập vào

☛ Ví dụ:

```
<input type="text" id="nameInput" />  
<button onclick="changeValue()">Đổi</button>  
<script>  
  function changeValue() {  
    document.getElementById("nameInput").value = "Nguyễn Văn A";  
  }  
</script>
```

5. So sánh nhanh

Thuộc tính	Mục đích	Dùng với	Ghi chú
<code>innerText</code>	Lấy/thay đổi text	Thế văn bản	Bỏ qua HTML con, không hiển thị thẻ
<code>innerHTML</code>	Lấy/thay đổi HTML	Thẻ chứa HTML	Cẩn thận với XSS
<code>value</code>	Lấy/thay đổi giá trị	input, textarea...	Chỉ áp dụng cho form controls

4. Gắn sự kiện: `onclick`, `addEventListener`

1. Khái niệm sự kiện (Event)

Sự kiện (event) là hành động của người dùng hoặc trình duyệt xảy ra trong trang web (click, nhập dữ liệu, cuộn trang, nhấn phím...)

- JavaScript có thể "nghe" và phản ứng lại các sự kiện này

2. Gắn sự kiện bằng onclick

☛ Cách 1: HTML inline

```
<button onclick="alert('Xin chào!')">Nhấn tôi</button>
```

☛ Cách 2: JavaScript

```
<button id="btn">Nhấn tôi</button>  
<script>  
  document.getElementById("btn").onclick = function() {  
    alert("Bấm vào nút!");  
  };  
</script>
```

☛ Lưu ý:

- `onclick` chỉ gắn được một sự kiện duy nhất
- Nếu gán lại `onclick`, sự kiện cũ sẽ bị ghi đè

3. Gắn sự kiện bằng `addEventListener`

☛ Cách dùng:

```
element.addEventListener("eventName", function);
```

Ví dụ:

```
<button id="btn">Click me</button>  
<script>  
  const btn = document.getElementById("btn");  
  btn.addEventListener("click", () => {  
    alert("Bấm đã click!");  
  });  
</script>
```

Ưu điểm:

- Có thể gán nhiều sự kiện cùng lúc
- Cho thể xóa sự kiện bằng `removeEventListener`
- Dễ kiểm soát hơn trong lập trình hiện đại

4. So sánh onclick vs `addEventListener`

Tiêu chí	<code>onclick</code>	<code>addEventListener</code>
Gắn 1 sự kiện duy nhất	☛	✗ (gắn nhiều được)
Đề dùng cho ví dụ nhỏ	☛	☛
Lập trình chuyên nghiệp	✗	☛
Có thể xóa sự kiện	✗	☛ Với <code>removeEventListener</code>

5. Một số sự kiện phổ biến khác

Sự kiện	Mô tả ngắn gọn
<code>click</code>	Khi người dùng nhấp chuột
<code>submit</code>	Khi form được gửi đi
<code>input</code>	Khi giá trị input thay đổi (liên tục)
<code>change</code>	Khi giá trị thay đổi (và rời focus)
<code>focus</code>	Khi input được focus vào
<code>blur</code>	Khi input bị bỏ focus

6. Bài tập thực hành

Đề bài: Đếm số lần click

```
<button id="countBtn">Click tôi</button>  
<p id="result">Số lần: 0</p>  
<script>  
  let count = 0;  
  const btn = document.getElementById("countBtn");  
  const res = document.getElementById("result");  
  
  btn.addEventListener("click", () => {  
    count++;  
    res.textContent = `Số lần: ${count}`;  
  });  
</script>
```

5. Storage (`localStorage`, `sessionStorage`, `cookies`)

1. Tổng quan về Web Storage

☛ Lý do cần Storage:

- Trình duyệt cần cách lưu dữ liệu nhỏ như:
- Trạng thái đăng nhập
- Cài đặt giao diện (dark mode)
- Thông tin giỏ hàng
- Token

☛ Có 3 loại lưu trữ:

Loại	Dung lượng	Thời gian sống	Tự động gửi về server?	Khả năng dùng
<code>localStorage</code>	~5MB	Vĩnh viễn (cho đến khi xóa)	✗	☛ Tốt cho dữ liệu client
<code>sessionStorage</code>	~5MB	Tồn tại khi tab còn mở	✗	☛ Dùng cho dữ liệu tạm
<code>Cookies</code>	~4KB	Do bạn đặt	☛ (gửi kèm mỗi request)	☛ Dùng cho xác thực

2. `localStorage`

☛ Cú pháp:

```
localStorage.setItem("key", "value");  
localStorage.getItem("key");  
localStorage.removeItem("key");  
localStorage.clear(); // Xóa toàn bộ
```

3. `sessionStorage`

☛ Tương tự `localStorage`, nhưng mất khi đóng tab

```
sessionStorage.setItem("cart", "3 items");  
sessionStorage.getItem("cart");  
sessionStorage.removeItem("cart");
```

Khi nào dùng?

Khi không muốn dữ liệu tồn tại lâu

Ví dụ: thông tin tạm trong 1 trang thanh toán

4. Cookies

☛ Cú pháp đặt cookie:

```
document.cookie = "username=John; expires=Fri, 31 Dec 2025 23:59:59 UTC; path=/";
```

☛ Đọc cookie:

```
console.log(document.cookie); // chuỗi dùng: "username=John; theme=dark"
```

☛ Xóa cookie:

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC;";
```

▲ Nhược điểm:

- Dung lượng nhỏ (~4KB)
- Gửi kèm mỗi HTTP request → ảnh hưởng hiệu suất

5. So sánh nhanh

Tiêu chí	<code>localStorage</code>	<code>sessionStorage</code>	<code>cookies</code>
Dung lượng	~5MB	~5MB	~4KB
Tồn tại bao lâu?	Vĩnh viễn	Khi tab còn mở	Tuỳ thời gian đặt
Gửi kèm request?	✗	✗	☛
Truy cập từ JS?	☛	☛	☛
Dùng cho đăng nhập?	Không nên	Không nên	☛ (với HttpOnly cookie)

6. Bài tập thực hành

- Tạo một button lưu người dùng vào `localStorage` và một button clear thông tin đó

Last changed by 



Quang Le Van · Follow

0 0 507 0

Add a comment